

SECTION 24 — DATABASE / SQL FOR SDET (Refined)

Topics Covered

- 24.1-24.23 (23 headers)

First pass — 2 parallel batch subagents

24.1 SQL Fundamentals — Refined

Theory: SQL is the standard language for relational databases — SDET's backend-validation tool. Core concepts: tables/rows/columns, primary keys (uniqueness), foreign keys (referential integrity), data types (`INT`, `VARCHAR`, `DATE`, `DECIMAL`), statement classes (DDL define, DML manipulate, DCL control), constraints (`NOT NULL`, `UNIQUE`, `CHECK`), ACID awareness for test oracles.

Enterprise Relevance: Every UI/API write ends in a DB row — SDET validates persistence, not just responses. ACID understanding separates "row exists" checks from correct transactional assertions.

```
CREATE TABLE Employees (
  id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  dept_id INT REFERENCES Departments(id)
);
```

High-Stakes Scenario: API returns 200 but row missing (async write failed silently) — UI-only suite green, data lost.

Triage: DB assertion after API (row exists + values), transaction boundary awareness, eventual-consistency polling where async.

Anti-Patterns: Asserting API response only (no persistence check); `SELECT *` in validation (brittle); no constraint awareness (orphans undetected).

Qs: DDL vs DML vs DCL with examples? What does a PK + FK guarantee that tests rely on?

24.2 SELECT / WHERE — Refined

Theory: `SELECT` retrieves columns; `WHERE` filters rows before any grouping occurs. Operators: `=`, `<>`, `>`, `<`, `BETWEEN`, `IN`, `LIKE`, `IS NULL` + logical `AND/OR/NOT` with parentheses for precedence. `DISTINCT` removes duplicates; `AS` aliases for readable validations. Core for verifying API → DB mappings (response field equals stored column).

Enterprise Relevance: 80% of SDET DB checks are `SELECT+WHERE` on IDs created via API. Correct `IS NULL` (never `=NULL`) and `IN` vs `OR` chains decide correctness.

```
SELECT id, name FROM Employees
WHERE dept_id IN (10, 20) AND name LIKE 'A%' AND status IS NOT NULL;
```

High-Stakes Scenario: Validation misses rows (`status = NULL` matches nothing — SQL three-valued logic). Triage: `IS NULL / IS NOT NULL` explicitly; `COALESCE` for nullable compares; row-count assertions alongside value assertions.

Anti-Patterns: `= NULL` ; `SELECT *` in checks; missing parentheses (`AND` binds tighter than `OR`); `LIKE '%...'` leading wildcard on huge tables in tests (slow).

Qs: Why does `WHERE status = NULL` return zero rows? `IN` vs chained `OR` ?

24.3 ORDER BY — Refined

Theory: `ORDER BY` sorts result sets deterministically for pagination and Top-N validation. Default ascending (`ASC`); specify `DESC` for descending. Sort by multiple columns, expressions, aliases, or ordinal positions. `NULLS FIRST/LAST` behavior varies (Oracle vs MySQL/Postgres) — specify explicitly in tests. Essential for sorted reports, leaderboards, recent-transaction scenarios.

Enterprise Relevance: Unordered assertions (`assertEquals(list)`) on unordered queries = order flake. Deloitte rule: any assertion on sequence requires explicit `ORDER BY` ; pagination tests assert page boundaries + total.

```
SELECT name, salary FROM Employees ORDER BY salary DESC, name ASC LIMIT 5;
-- recent-first check:
SELECT id FROM orders ORDER BY created_at DESC NULLS LAST LIMIT 1;
```

High-Stakes Scenario: "Latest order" test without `ORDER BY` passes 90% (index order coincidence), fails after vacuum/reindex. Triage: always `ORDER BY created_at DESC, id DESC` ; never rely on insertion order; pagination tests assert overlapping boundaries.

Anti-Patterns: No `ORDER BY` with `LIMIT` ; assuming insertion order; `NULLS` behavior assumed cross-DB; ordinal positions (`ORDER BY 2`) in maintained code.

Qs: Why is `LIMIT` without `ORDER BY` non-deterministic? How do `NULLs` sort in your DB?

24.4 GROUP BY — Refined

Theory: `GROUP BY` collapses rows sharing column values into summary groups, typically with aggregates. All non-aggregated `SELECT` columns must appear in `GROUP BY` (strict SQL; MySQL lenient mode hides bugs). Execution order: `WHERE > GROUP BY > HAVING > ORDER BY` — memorize it, interviews ask. Use for counts per status, department headcounts, duplicate detection. Combine with `DISTINCT` aggregates when needed.

Enterprise Relevance: Validation queries for dashboards ("orders per status must match UI counts") are `GROUP BY` queries. Wrong grouping = wrong expected values = false failures blamed on product.

```
SELECT dept_id, COUNT(*) AS headcount FROM Employees GROUP BY dept_id;
-- duplicate detection:
SELECT email, COUNT(*) c FROM users GROUP BY email HAVING COUNT(*) > 1;
```

High-Stakes Scenario: MySQL lenient `GROUP BY` returns arbitrary row values; same suite fails on Postgres (strict). Triage: write portable SQL (all non-agg in `GROUP BY`), test against CI DB engine (not just local MySQL), `ONLY_FULL_GROUP_BY` on.

Anti-Patterns: Non-aggregated `SELECT` cols missing from `GROUP BY`; `WHERE` on aggregate (use `HAVING`); grouping by display alias unsupported in some DBs.

Qs: Recite execution order 1-8? Why does MySQL accept invalid `GROUP BY` that Postgres rejects?

24.5 HAVING — Refined

Theory: **HAVING** filters grouped results after aggregation, whereas **WHERE** filters individual rows before grouping. Always use **HAVING** with aggregate conditions (`COUNT(*) > 1`). It can reference aliases in MySQL but use expressions for Oracle portability. Critical SDET checks: duplicate orders, departments exceeding budget, customers with multiple failed logins.

Enterprise Relevance: Data-quality assertions (no duplicates, threshold breaches) are HAVING queries. **WHERE COUNT(*)** is the classic syntax error that reveals SQL inexperience in live coding.

```
SELECT dept_id, COUNT(*) AS c FROM Employees GROUP BY dept_id HAVING COUNT(*) > 5;
-- departments over budget:
SELECT dept_id, SUM(salary) s FROM Employees GROUP BY dept_id HAVING SUM(salary) > 1000000;
```

High-Stakes Scenario: Duplicate-order bug — validation used `WHERE count > 1` (error) so check never ran; dupes shipped. Triage: HAVING for post-aggregation; run data-quality suite (dup/orphan/null checks) nightly, not just functional asserts.

Anti-Patterns: **WHERE** with aggregates; alias in HAVING on Oracle; HAVING without GROUP BY (valid but confusing — whole result is one group).

Qs: WHERE vs HAVING with execution order? Write duplicate-detection query from memory?

24.6 Aggregate Functions — Refined

Theory: Aggregates compute single values from sets: **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**. **COUNT(*)** counts rows including nulls; **COUNT(col)** ignores NULLs. **SUM/AVG** skip nulls, affecting expected calculations. Use **COALESCE** to handle nulls, **ROUND** for decimals. Vital for reconciling UI totals, invoice sums, report dashboards against database truth.

Enterprise Relevance: Money assertions live here — `SUM(line_total)` vs UI total; off-by-null (skipped NULLs) vs off-by-penny (**ROUND**) are the two classic finance-test failures.

```
SELECT dept_id, COUNT(*) AS n, AVG(salary) AS avg_sal, MAX(salary) AS top
FROM Employees GROUP BY dept_id;
-- null-safe revenue:
SELECT COALESCE(SUM(amount), 0) FROM payments WHERE status = 'PAID';
```

High-Stakes Scenario: UI total \$10,000 vs DB `SUM` \$9,800 — 2 voided rows with NULL amounts skipped by SUM but UI counts them as 0. Triage: `COALESCE(amount, 0)` alignment, voided-row handling documented, penny-rounding `ROUND(..., 2)` agreed with finance.

Anti-Patterns: **COUNT(col)** when nulls matter; **AVG** on nullable without **COALESCE** note; float sums for money (use **DECIMAL**); asserting exact **AVG** across DBs (rounding differs).

Qs: **COUNT(*)** vs **COUNT(col)** vs **COUNT(DISTINCT col)** ? How do NULLs affect SUM/AVG?

24.7 Joins — Refined

Theory: Joins combine rows from multiple tables via related keys using **ON** predicates. Understand Venn-diagram semantics, cardinality one-to-many, and impact of duplicate keys causing fan-out row multiplication (2 orders × 3 items = 6 rows — correct, but **COUNT(*)** surprises). Always qualify columns (`e.name`, `d.name`) with aliases. For SDET:

joins validate end-to-end referential integrity across normalized microservice schemas and ETL mappings.

Enterprise Relevance: Cross-service validation (order + customer + payment) is JOIN validation. Fan-out double-counting is the #1 wrong-expected-value cause in report tests.

```
SELECT e.name, d.dept_name FROM Employees e JOIN Departments d ON e.dept_id = d.id;
-- fan-out aware counting:
SELECT COUNT(DISTINCT o.id) FROM orders o JOIN items i ON i.order_id = o.id;
```

High-Stakes Scenario: Revenue report test expects \$50k, query returns \$150k — join fan-out (3 items/order × sum over joined rows). Triage: aggregate before joining (subquery/CTE per entity), `COUNT(DISTINCT)`, or validate at grain (per-order) not rolled-up.

Anti-Patterns: Comma joins (`FROM a, b WHERE`) — use explicit `JOIN...ON`; unqualified columns (ambiguous); joining on non-unique keys without fan-out awareness; `SELECT *` across joins (column collisions).

Qs: Why does joining inflate counts? How do you validate one-to-many without double-counting?

24.8 Inner Join — Refined

Theory: `INNER JOIN` returns only matching rows in both tables — the default join type. Use for happy-path referential data (orders having valid customers). Non-matching rows from either side are excluded, so missing data indicates orphans. Test with `JOIN...ON` versus deprecated comma syntax. Check for unintended duplicates when join keys are non-unique.

Enterprise Relevance: Inner join validates the "everything linked correctly" path. Complement with `LEFT JOIN` orphan checks (24.9) — inner proves presence, left-anti proves absence of orphans.

```
SELECT o.id, c.name FROM Orders o INNER JOIN Customers c ON o.customer_id = c.id;
-- orphan count (should be 0 for enforced FKs):
SELECT COUNT(*) FROM Orders o LEFT JOIN Customers c ON o.customer_id = c.id WHERE c.id IS NULL;
```

High-Stakes Scenario: Migrated orders reference deleted customers — inner join silently drops 200 rows, report total mismatches UI count. Triage: row-count reconciliation (source vs joined), orphan query in data-quality suite, FK constraint verification post-migration.

Anti-Patterns: Assuming inner join preserves row counts; comma syntax; no orphan counterpart check.

Qs: What rows does `INNER JOIN` drop, and why does that matter for validation?

24.9 Left Join — Refined

Theory: `LEFT JOIN` returns all left-table rows plus matched right-table rows, with `NULL` s where no match exists. Perfect for completeness audits: employees without departments, orders without payments. Filter with `WHERE right.id IS NULL` (left-anti pattern) to isolate orphans. For testing: assert optional relationships and verify UI handles null-joined fields gracefully without dropping records.

Enterprise Relevance: The orphan-finding pattern (`LEFT JOIN...IS NULL`) is a data-quality staple — run it nightly over FK relationships. Every nullable relationship needs both a happy-path (inner) and orphan (left-anti) check.

```
SELECT e.name, d.dept_name FROM Employees e LEFT JOIN Departments d ON e.dept_id = d.id;
-- orphans: orders with no payment
SELECT o.id FROM orders o LEFT JOIN payments p ON p.order_id = o.id WHERE p.id IS NULL;
```

High-Stakes Scenario: UI drops orders with missing optional `promo_code` (inner join in app query) — 5% orders invisible. Triage: left-join audit finds orphans, app query fixed to `LEFT`, UI null-handling test added (renders "—" not blank/crash).

Anti-Patterns: `WHERE` on right-table column (converts `LEFT` to `INNER` — put filters in `ON`); no orphan checks; UI assuming non-null joined fields.

Qs: `LEFT JOIN + WHERE right IS NULL` = what? Why does `WHERE` on right table break `LEFT` semantics?

24.10 Right Join — Awareness — Refined

Theory: `RIGHT JOIN` mirrors `LEFT JOIN`, preserving all right-table rows. Rare in production — most teams rewrite as `LEFT JOIN` (swap table order) for readability. Awareness matters because Oracle reports and legacy queries may use it. `WHERE left.id IS NULL` finds right-only orphans. Interview tip: state preference for `LEFT JOIN` while demonstrating ability to interpret both.

Enterprise Relevance: You'll read `RIGHT JOIN` in legacy code, never write it in new tests. Rewrite on sight for reviewer sanity.

```
-- legacy RIGHT (preserve departments even w/o employees):
SELECT e.name, d.dept_name FROM Employees e RIGHT JOIN Departments d ON e.dept_id = d.id;
-- preferred rewrite:
SELECT e.name, d.dept_name FROM Departments d LEFT JOIN Employees e ON e.dept_id = d.id;
```

High-Stakes Scenario: Legacy report uses `RIGHT JOIN`; new hire "simplifies" to `INNER` — departments with zero headcount vanish from compliance report. Triage: row-count parity check on any join-type change; empty-side preservation tests.

Anti-Patterns: Writing new `RIGHT JOINS`; converting `RIGHT` → `INNER` accidentally; not testing empty-side rows.

Qs: Rewrite `RIGHT` as `LEFT`? When must empty-side rows be preserved?

24.11 Self Join — Refined

Theory: Self join joins a table to itself using two aliases to model hierarchies or sequential comparisons. Classic SDET uses: employee-manager trees, duplicate-email detection, comparing rows within the same table (e.g., current vs previous salary). Requires careful aliasing (`e1`, `e2`) to avoid ambiguity. Combine with `LEFT JOIN` to include top-level nodes (CEO with no manager).

Enterprise Relevance: Hierarchy validation (org charts, category trees, threaded comments) and duplicate detection are self-join queries. Alias discipline is what interviewers watch.

```
-- employee → manager (CEO included via LEFT):
SELECT e.name AS emp, m.name AS manager
FROM Employees e LEFT JOIN Employees m ON e.manager_id = m.id;
-- duplicate emails:
SELECT e1.email FROM Employees e1 JOIN Employees e2
ON e1.email = e2.email AND e1.id < e2.id;
```

High-Stakes Scenario: Manager Cascading delete test — self-referencing FK with `ON DELETE CASCADE` wipes entire subtree (deleted manager + all reports). Triage: hierarchy-aware fixtures (known depth), cascade-rule verification, orphan/reparent checks post-delete.

Anti-Patterns: Missing aliases (ambiguous column); `=` instead of `<` in dup detection (self-matches); `INNER`

excluding top-level nodes unintentionally.

Qs: Write employee-manager query including CEO? How do you find duplicates with self-join?

24.12 Subqueries — Refined

Theory: Subqueries nest `SELECT` inside `SELECT/WHERE/FROM/HAVING` for stepwise logic. Types: scalar (single value, `=`), single-row, multi-row (`IN`, `ANY`, `ALL`), correlated (references outer query, runs per row) vs non-correlated (runs once), derived tables (`FROM (SELECT...)` with mandatory alias). Use `EXISTS` for efficient existence checks over `IN` with nulls (`IN` with `NULL` in list returns unknown, silently dropping rows). Ideal for SDET validation when joins get complex: second-highest salary, customers without orders.

Enterprise Relevance: Subqueries express validation intent readably ("customers with no orders") where joins obscure it. `NOT IN` vs `NOT EXISTS` null trap has caused real false-green validations.

```
-- second-highest salary:
SELECT name, salary FROM Employees
WHERE salary = (SELECT MAX(salary) FROM Employees WHERE salary < (SELECT MAX(salary) FROM Employees))
-- customers with no orders (null-safe):
SELECT c.id FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id)
```

High-Stakes Scenario: `NOT IN (SELECT customer_id ...)` with one `NULL` in subquery → returns zero rows always (`NULL` poisons `IN`) → "no orphan customers" check falsely green while orphans exist. Triage: `NOT EXISTS` (null-safe) or `NOT IN + WHERE col IS NOT NULL`; mutation test (insert orphan, check must fail).

Anti-Patterns: `NOT IN` with nullable subquery; correlated subquery on huge tables (runs per row — use `JOIN`); unnamed derived tables (syntax error); scalar subquery returning multiple rows.

Qs: `NOT IN` vs `NOT EXISTS` with `NULL`s? Correlated vs non-correlated performance?

24.13 CTE (Common Table Expressions) — Refined

Theory: A CTE is a named temporary result set defined with `WITH` for one statement — improving readability over nested subqueries. Break complex validation queries into named steps. Supports chaining (multiple CTEs) and recursion (hierarchies with `UNION ALL`). Scope ends after the main query; can be referenced multiple times in `SELECT/JOIN` (unlike derived tables used once).

Enterprise Relevance: Validation queries with 3+ nesting levels become unreviewable; CTEs name each step (readable, debuggable, reusable within the query). Deloitte data-quality suites standardize on CTEs for multi-step checks.

```
WITH HighPaid AS (
  SELECT dept_id, AVG(salary) AS avg_sal FROM emp GROUP BY dept_id
)
SELECT * FROM HighPaid WHERE avg_sal > 80000;
-- recursive hierarchy:
WITH RECURSIVE reports AS (
  SELECT id, manager_id, 1 lvl FROM emp WHERE id = 7
  UNION ALL
  SELECT e.id, e.manager_id, r.lvl + 1 FROM emp e JOIN reports r ON e.manager_id = r.id
) SELECT * FROM reports;
```

High-Stakes Scenario: 5-level nested subquery validation fails — nobody can tell which level broke. Triage: rewrite as chained CTEs (one concept per CTE), test each CTE standalone (`SELECT * FROM step2`), name steps by business meaning.

Anti-Patterns: Nesting beyond 2 levels (use CTE); CTE referenced once (fine, but derived table equivalent); recursive CTE without depth guard (infinite loop on cyclic data); assuming CTE materializes (optimizers may inline — perf differs by DB).

Qs: CTE vs subquery vs temp table? When does recursion need a depth guard?

24.14 Window Functions — Refined

Theory: Window functions compute across related rows without collapsing them (unlike `GROUP BY` which reduces to one row per group). Syntax: `fn() OVER(PARTITION BY ... ORDER BY ... ROWS/RANGE ...)`. Categories: ranking (`ROW_NUMBER/RANK/DENSE_RANK`), aggregate (`SUM/AVG/COUNT` running), value (`LAG/LEAD/FIRST_VALUE/LAST_VALUE`). Essential for running totals, moving averages, Top-N per group. Evaluated after `WHERE/GROUP BY`, before `ORDER BY` — hence cannot appear in `WHERE` (wrap in subquery/CTE to filter). Enterprise Relevance: Per-group Top-N, running totals, period-over-period — the staple of report validation. Windows express them in one pass where self-joins would be $O(n^2)$.

```
SELECT name, dept, salary,
       AVG(salary) OVER (PARTITION BY dept) AS dept_avg,
       SUM(salary) OVER (ORDER BY salary ROWS UNBOUNDED PRECEDING) AS running_total,
       LAG(salary) OVER (PARTITION BY dept ORDER BY hired) AS prev_hire_sal
FROM emp;
```

High-Stakes Scenario: `WHERE RANK()...` syntax error in validation (windows can't be in `WHERE`). Triage: wrap — `SELECT * FROM (SELECT ..., RANK() OVER(...) r FROM t) WHERE r <= 3`; CTE form for readability; `LAST_VALUE` trap needs explicit frame (`UNBOUNDED FOLLOWING`) or returns current row.

Anti-Patterns: Window in `WHERE/GROUP BY`; missing frame with `LAST_VALUE`; `ORDER BY` omitted in ranking (nondeterministic); confusing `PARTITION` (grouping) with `GROUP BY` (collapsing).

Qs: Why can't windows appear in `WHERE`? `PARTITION BY` vs `GROUP BY`?

24.15 ROW_NUMBER — Refined

Theory: `ROW_NUMBER()` assigns unique sequential integers starting at 1 within each partition, ordered by `OVER(ORDER BY ...)`. Ties get arbitrary different numbers (nondeterministic among equals — unlike `RANK/DENSE_RANK` which share). Uses: deduplication (keep `rn=1`), pagination, latest record per group, duplicate deletion via CTE. Requires outer filter since window functions cannot appear in `WHERE`.

Enterprise Relevance: "Latest X per Y" (latest order per customer, current salary per employee) is `ROW_NUMBER rn=1`. Dedup cleanup (`DELETE WHERE rn>1`) is a data-hygiene staple.

```
-- latest user row per email:
SELECT * FROM (
  SELECT id, email, created_at,
         ROW_NUMBER() OVER (PARTITION BY email ORDER BY created_at DESC) AS rn
  FROM users
) t WHERE rn = 1;
-- delete duplicates, keep earliest:
WITH d AS (
```

```
SELECT id, ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) rn FROM users
) DELETE FROM users WHERE id IN (SELECT id FROM d WHERE rn > 1);
```

High-Stakes Scenario: Pagination with ROW_NUMBER but unstable ORDER BY (ties arbitrary) — rows appear on multiple pages / vanish between pages. Triage: deterministic tiebreak (ORDER BY created_at DESC, id DESC — unique suffix), keyset pagination for large sets.

Anti-Patterns: No tiebreak (nondeterministic rn among ties); ROW_NUMBER for Top-N distinct (use DENSE_RANK); filtering rn in same query level (wrap it).

Qs: ROW_NUMBER vs RANK vs DENSE_RANK on ties? Why must rn filtering wrap in subquery?

24.16 RANK — Refined

Theory: RANK() assigns the same rank to tied ORDER BY values but skips subsequent ranks: salaries 100k, 90k, 90k, 80k → ranks 1, 2, 2, 4. Use when business ranking must reflect gaps (competition ranking, salary bands — "two people tied 2nd, next is 4th"). Compare with ROW_NUMBER (always unique) in interviews. Often combined with PARTITION BY dept for per-group ranking.

Enterprise Relevance: Leaderboards, salary bands, competition results. The gap behavior is the business rule — using ROW_NUMBER here misranks (shows 3rd where business says 4th).

```
SELECT name, salary, RANK() OVER (ORDER BY salary DESC) AS rnk FROM emp;
-- per-department ranking:
SELECT name, dept, salary, RANK() OVER (PARTITION BY dept ORDER BY salary DESC) AS dept_rank FROM emp;
-- salaries 100k,90k,90k,80k => ranks 1,2,2,4
```

High-Stakes Scenario: Bonus eligibility "top 3 ranks" — RANK gives 4 people (1,2,2,4 includes rank 4? No: ranks ≤3 = 3 people: 1,2,2). Triage: clarify business definition (top-3 ranks vs top-3 people vs distinct levels) — RANK vs DENSE_RANK vs ROW_NUMBER give different sets with ties; document which.

Anti-Patterns: RANK for Top-N people with ties (ambiguous count); no PARTITION when per-group needed; assuming rank values contiguous.

Qs: RANK 1,2,2,? — what comes next and why? When does the gap matter?

24.17 DENSE_RANK — Refined

Theory: DENSE_RANK() gives ties the same rank but never gaps: 100k, 90k, 90k, 80k → 1, 2, 2, 3. Best for Nth-highest distinct salary and Top-N distinct levels. Deloitte favorite: second-highest salary without missing rank 3 after ties (RANK would give 1,2,2,4 — no rank 3 exists). Use PARTITION BY for per-department Top-N.

Enterprise Relevance: "Second-highest salary" interview + real pay-band validation. DENSE_RANK = distinct levels; the correct answer when business asks for distinct values.

```
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS dr FROM emp
) t WHERE dr = 2; -- second distinct level, always exists if ≥2 levels
-- top-2 distinct per dept:
SELECT * FROM (
  SELECT *, DENSE_RANK() OVER (PARTITION BY dept ORDER BY salary DESC) dr FROM emp
) t WHERE dr <= 2;
```

High-Stakes Scenario: "Top 3 earners" with ties — ROW_NUMBER returns 3 people (arbitrary tiebreak), RANK may return 2 or 5, DENSE_RANK returns all in top-3 levels. Triage: ask business which semantic (people vs levels); document choice; test tie cases explicitly.

Anti-Patterns: ROW_NUMBER for distinct-level questions; RANK expecting contiguous ranks; no tie test data (all salaries unique → functions indistinguishable).

Qs: ROW_NUMBER vs RANK vs DENSE_RANK on 100,90,90,80? Which answers "2nd highest distinct"?

24.18 Transactions — Refined

Theory: A transaction is an atomic unit satisfying ACID: Atomicity (all-or-nothing), Consistency (valid state to valid state), Isolation (concurrent txns don't corrupt), Durability (committed survives crashes). **BEGIN/START TRANSACTION** groups operations (fund-transfer debit + credit both succeed or none). SDET relevance: test rollback on failure, isolation anomalies (dirty/non-repeatable/phantom reads), savepoints for partial rollback. Oracle auto-commits DDL; failed statements may abort the whole block.

Enterprise Relevance: Money movement, order creation (order + items + payment), user onboarding (user + profile + entitlements) — all multi-statement units. Tests must verify atomicity: kill mid-transaction → nothing partial persists.

```
BEGIN;
UPDATE accounts SET bal = bal - 100 WHERE name = 'Alice';
UPDATE accounts SET bal = bal + 100 WHERE name = 'Bob';
COMMIT; -- or ROLLBACK on any error
```

High-Stakes Scenario: Transfer debits Alice, credit fails (constraint) — without transaction, money vanishes. Triage: wrap in txn, test failure injection (kill after debit), assert balances unchanged + error surfaced; check isolation level (read-committed vs serializable) for concurrent transfers.

Anti-Patterns: Autocommit multi-statement business ops; no rollback test; testing only happy commit; ignoring isolation levels under concurrency.

Qs: ACID letters with test example each? What happens to uncommitted work on crash?

24.19 Commit / Rollback — Refined

Theory: **COMMIT** permanently saves transaction changes, releases locks, erases savepoints, makes data visible to others. **ROLLBACK** undoes all uncommitted changes; **ROLLBACK TO SAVEPOINT** undoes partially (named checkpoints within txn). MySQL defaults **autocommit=1** (each statement commits); use **SET autocommit=0** or explicit **START TRANSACTION**. Always assert DB state after both paths (commit persists, rollback pristine).

Enterprise Relevance: Cleanup and negative tests depend on rollback (no residue). Autocommit surprises (DDL committing in Oracle, MySQL single-statement commits) break test isolation assumptions.

```
START TRANSACTION;
INSERT INTO orders(id, total) VALUES(101, 99.00);
SAVEPOINT s1;
UPDATE orders SET total = 0 WHERE id = 101; -- oops
ROLLBACK TO s1; -- undo update, keep insert
COMMIT; -- persist insert
-- test cleanup pattern:
START TRANSACTION; /* test writes */ ROLLBACK; /* pristine */
```

High-Stakes Scenario: Test cleanup via ROLLBACK fails — MySQL DDL (CREATE TEMPORARY) auto-committed mid-test, partial data persists, next test collides. Triage: know auto-commit triggers per DB (DDL!), avoid DDL in tests, explicit transaction control, verify post-rollback counts.

Anti-Patterns: Assuming ROLLBACK undoes DDL (Oracle/MySQL); no savepoints for multi-step setup; asserting only commit path; autocommit left on in transaction tests.

Qs: What does COMMIT release besides data? When does ROLLBACK fail to restore?

24.20 Primary Key / Foreign Key — Refined

Theory: **PRIMARY KEY** uniquely identifies each row: **NOT NULL**, one per table, auto-creates unique (clustered) index. **FOREIGN KEY** references parent PK/UNIQUE, enforcing referential integrity and preventing orphans. Actions: **ON DELETE CASCADE** (delete children), **SET NULL**, **RESTRICT** (block). Validate constraints in automation via duplicate, null, and orphan inserts (expect constraint violations, not silent acceptance).

Enterprise Relevance: Constraint tests prove the schema guards data when app code fails. Every FK needs: valid insert passes, orphan insert rejected, parent delete follows declared action (cascade/null/restrict verified).

```
CREATE TABLE orders(
  o_id INT PRIMARY KEY,
  u_id INT,
  CONSTRAINT fk_o_u FOREIGN KEY(u_id) REFERENCES users(u_id)
  ON DELETE CASCADE
);
-- negative tests:
INSERT INTO orders VALUES(1, 99999); -- orphan → must fail FK violation
INSERT INTO orders VALUES(1, 1); INSERT INTO orders VALUES(1, 2); -- dup PK → must fail
```

High-Stakes Scenario: Missing FK (app-enforced only) — bug deletes user, 500 orphan orders break reports for weeks before detection. Triage: schema audit (every relationship has FK?), orphan queries nightly, constraint-violation negative tests, cascade behavior verified per relationship.

Anti-Patterns: App-only integrity (no DB constraints); untested CASCADE (surprise mass deletes); nullable FK without null-handling tests.

Qs: PK vs UNIQUE vs FK? What do CASCADE/SET NULL/RESTRICT do on parent delete?

24.21 Indexes — Interview Depth (B-Tree lookups and indexing overhead) — Refined

Theory: B-Tree is the default balanced-tree index: root, branch, leaf pages kept sorted, enabling $O(\log n)$ search, range scans, and ordering. Dramatically speeds **WHERE/JOIN/ORDER BY** but slows **INSERT/UPDATE/DELETE** (index maintenance) and uses space. Create on selective filter/FK columns; avoid over-indexing (each index taxes writes). Verify with **EXPLAIN** (check index usage, not just existence).

Enterprise Relevance: SDET doesn't design indexes but must read **EXPLAIN** output (is my validation query using an index or full-scanning 10M rows and timing out the suite?) and test index-backed constraints (UNIQUE violations).

```
CREATE INDEX idx_emp_dept_sal ON emp(dept_id, salary);
EXPLAIN SELECT * FROM emp WHERE dept_id = 10 ORDER BY salary;
-- look for: Index Scan using idx_emp_dept_sal (good) vs Seq Scan (bad at scale)
```

High-Stakes Scenario: Validation query full-scans 50M-row table per test → 30s each × 100 tests = suite timeout.

Triage: EXPLAIN, composite index on (filter + order) columns, assert on indexed columns, limit validation scope (partition/run_id), separate validation replica.

Anti-Patterns: Functions on indexed columns (WHERE YEAR(d)=2024 kills index — use range); leading %LIKE% (unindexable); over-indexing write-heavy tables; assuming index = always faster (small tables: seq scan wins).

Qs: How does a B-Tree turn O(n) into O(log n)? When does an index hurt?

24.22 Database Validation in Automation — Refined

Theory: Backend validation confirms UI/API writes persisted correctly via JDBC/DB clients. Pattern: create order via API → query DB with PreparedStatement (never string concat — SQL injection even in tests) → assert status, totals, audit rows, soft-delete flags. Handle eventual consistency with polling (not sleep), separate test data (run_id scoping), cleanup transactions. Check orphans, counts, constraints — not just the happy row.

Enterprise Relevance: This closes the loop UI → API → DB (Sec 20.18). A green API test with missing DB row is a false pass; DB validation catches async-write failures, trigger bugs, and mapping errors.

```
// JDBC validation with polling for eventual consistency
try (Connection c = dataSource.getConnection();
    PreparedStatement ps = c.prepareStatement("SELECT status,total FROM orders WHERE id=?")) {
    ps.setLong(1, orderId);
    Order row = await().atMost(10, SECONDS).until(() -> queryOne(ps), Objects::nonNull);
    assertEquals("PAID", row.status());
}
-- orphans check (data-quality):
SELECT o.id FROM orders o LEFT JOIN users u ON o.u_id = u.u_id WHERE u.u_id IS NULL;
```

High-Stakes Scenario: Async projection lags 5s — DB assert immediately after API fails intermittently. Triage: polling wrapper (awaitility, timeout + interval), idempotent assertions, separate eventual (poll) vs immediate (direct) checks by architecture.

Anti-Patterns: String-concat SQL; no polling on async writes; shared-table assertions (collisions); no cleanup (row accumulation); asserting counts without scoping (other tests' rows).

Qs: How do you validate async writes deterministically? What injection risk exists in test SQL?

24.23 SQL Coding Questions — Refined

Deloitte SDET favorites: second/Nth salary, Top-N per department, duplicates, orphans, joins. Master DENSE_RANK vs ROW_NUMBER, LIMIT/OFFSET, MAX-where-less-than-MAX, GROUP BY/HAVING, LEFT JOIN...IS NULL. Always handle NULL, ties, empty result (returns NULL, not error).

```
-- 2nd highest (3 ways — know all):
SELECT MAX(salary) FROM emp WHERE salary < (SELECT MAX(salary) FROM emp);
SELECT salary FROM (SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) dr FROM emp) t WHERE
SELECT DISTINCT salary FROM emp ORDER BY salary DESC LIMIT 1 OFFSET 1;
-- Top-2 per dept:
SELECT * FROM (SELECT *, ROW_NUMBER() OVER (PARTITION BY dept ORDER BY salary DESC) rn FROM emp)
-- duplicates:
SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1;
```

```
-- orphans:  
SELECT o.id FROM orders o LEFT JOIN users u ON o.u_id = u.u_id WHERE u.u_id IS NULL;
```

Triage: empty table → aggregates return NULL (assert accordingly); ties → specify RANK semantic; **LIMIT** without **ORDER BY** (nondeterministic).

Anti-Patterns: **LIMIT 1,1** MySQL-only in portable code; ignoring ties; no NULL/empty-case handling.

Qs: Three ways to Nth salary + trade-offs? Top-N per group pattern from memory?